Roy and Coin Boxes - Detailed Explanation

🔥 Problem Statement

Roy ke paas **N** coin boxes hain, aur starting mein sabhi boxes ekdum khaali (0 coins) hain. Uske baad **M** dino tak, roz ek task hota hai: Aapko ek range di jayegi `[L, R]`, aur aapko **L se lekar R tak ke har box mein 1 coin daalna hai**.

Jab M dino ka task khatam ho jayega, tab Roy aapse **Q** number of queries (sawal) pucha karega. Har sawal mein ek number **X** hoga, aur aapko batana hai:

> ***"Aise kitne boxes hain jinme kam se kam (at least) X coins rakhe hue hain?"***

Example se samjho:

Maan lo **7 boxes** hain (`N = 7`).

- Din 1: range `[1, 3]` ➔ Box 1, 2, 3 mein +1 coin
- Din 2: range `[2, 5]` ➔ Box 2, 3, 4, 5 mein +1 coin
- Din 3: range `[1, 2]` ➔ Box 1, 2 mein +1 coin
- Din 4: range `[5, 6]` ➔ Box 5, 6 mein +1 coin

Sab khatam hone ke baad boxes ka haal:

- Box: `1 2 3 4 5 6 7`
- Coins: `2 3 2 1 2 1 0`

Agar query aaye `X = 2` : Aise kitne boxes hain jisme **2 ya 2 se zyada** coins hain? Ans: Box 1, 2, 3 aur 5. Toh answer hai **4**.

🧠 Brute Force Approach (Kyun nahi chalega?)

Agar hum har din ek loop chalayein `for(int i = L; i <= R; i++) { boxes[i]++; }`, toh time complexity `O(N * M)` ho jayegi. Constraints mein $N = 10^6$ aur $M = 10^6$ hai. Toh total operations 10^{12} ban jayenge, jiska matlab seedha **Time Limit Exceeded (TLE)** aayega. Hume ise `O(N)` mein solve karna hoga.

🚀 Optimized Approach (The O(N) Logic)

Is problem ko effectively 4 parts mein divide karke solve karna hai.

Step 1: Range Queries using Difference Array Concept

Bar bar `L` se `R` tak loop chalane ki jagah hum sirf array ki "boundaries" pe nishaan lagayenge. Hum 2 arrays banayenge:

1. `L_array` : Ye batayega ki index i pe kitne intervals START ho rahe hain.
2. `R_array` : Ye batayega ki index i pe kitne intervals END ho rahe hain.

Kaise karein? Har range `[L, R]` ke liye: `l[L]++` (Matlab L se ek continuous addition shuru hua) `r[R]++` (Matlab R pe wo addition khatam ho gaya)

Step 2: Har box me final coins calculate karna (Prefix Sum formula)

Ab hume har box `i` ke exact coins nikalne hain. Ab socho, kisi box mein kitne coin honge?

> ***Formlua:*** *Aaj ke coins = Pichle wale box ke coins + Naye coins jo isi box se start hue - Wo coins jinki range pichle wale box me hi khatam ho chuki hai.*

Code mein iska logic hai: `f[i] = f[i-1] + l[i] - r[i-1]`

Logic detail: Hum `l[i]` isliye add karte hain kyunki koi nayi range `i` se chalu hui hai toh usme 1 coin badh jayega. `r[i-1]` isliye minus karte hain kyunki jo range `i-1` pe khatam ho gayi, wo `i` tak aayegi hi nahi, isliye use sum me se nikalna padega.

Step 3: Frequency Array Banana (Frequency of coins)

Hume queries solve karni hain ">= X" wali. Us se pehle hume pata hona chahiye ki **Exactly X** coins kitne box mein hain. Iske liye ek `freq` array banate hain. `freq[c]` bolega: "Total kitne boxes aise hain jinke paas exactly `c` coins hain".

Loop lagake: `freq[ f[i] ]++` kardo. E.g., Agar box-2 mein 3 coins hain, toh `freq[3]` pe count badh jayega.

Step 4: Suffix Sum for Suffix Frequency

Hume exact frequency nahi chahiye, humari test queries mangti hain `AT LEAST X` (>=X) coins. Agar mujhe `>= 5` coins wale boxes chahiye toh: `Mera Answer = freq[5] (exactly 5) + freq[6] + freq[7] + ... aage wale sabhi`

Aage walo ka sum baar baar loop me na nikalna pade isliye hum **Suffix Sum** (Ulta Prefix sum) karenge.

> *`suffix_sum[i] = suffix_sum[i+1] + freq[i]` Ye array hume peeche se jod ke answer de dega. Ab agar query puche `X = 2`, toh hum answer directly `suffix_sum[2]` print kar denge O(1) me!*

💻 Code Architecture Recap

1. Array Size `N+2` lenge takki Out of Bound / Seg Fault issues na aaye `r[i-1]` type operations me.
2. Sab inputs leke boundaries update.
3. Ek pass Prefix calculation for mapping coins in boxes `f[]`.
4. Ek pass Frequency nikalne ke liye `freq[]`.
5. Ek pass Ulta loop (Suffix Sum) frequencies add karne ke liye `suffix[]`.
6. Queries ko O(1) me print karo.

Hinglish aur aisi detailed strategy se is sawal ko kabhi bhuloge nahi! 🤘